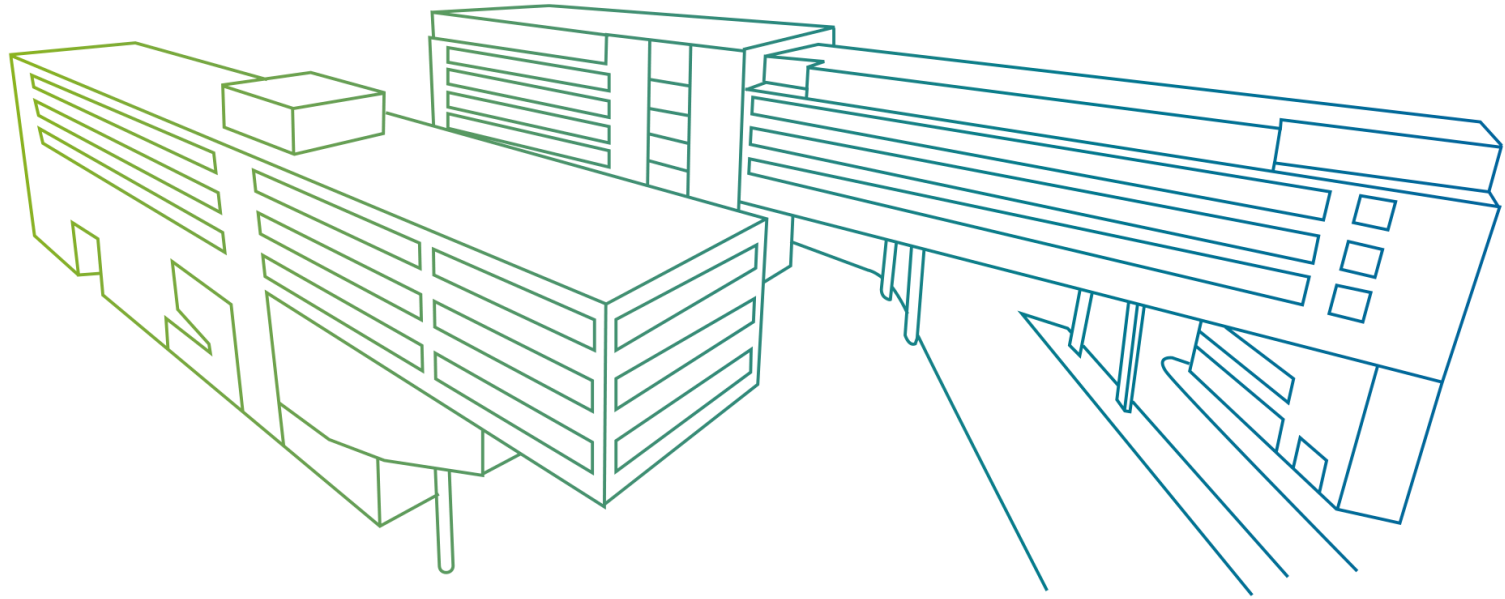




Probabilistic Robotics

Sensor & Motion Model
Lucas Muster, MSc

The Problem so far

- We assumed a lot of things
 - The linear motion is $\vec{x}_t = A\vec{x}_t + B\vec{u}_t$ (Similar problems in the EKF)
 - Linear sensor model: $C\vec{x}_t^*$
 - ...
- But until now we have not...

... used real sensor noise.

→ We applied a general statistical model to describe the sensor's uncertainty. Again, the uncertainty calculation is strongly connected to the kind of sensor.

... used the real kinematic model.

→ The uncertainty using Σ_t describes a general motion and not a motion of a real mobile robot. The wheel motion is noisy. This noise has to be "moved" through the kinematic model. So, what we need to do is implementing sensor and motion models for real hardware.

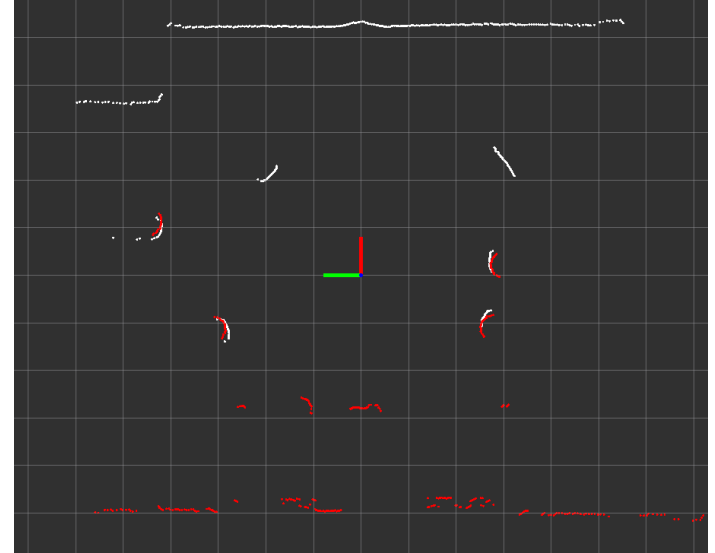
So, what we need to do is implementing sensor and motion models for real hardware.

How to Overcome the Problem?

- Sensor model
 - We will describe the sensor behavior
 - In this lecture, range finder (2D scanner)
- Motion model
 - We describe noise where it belongs (wheels)
 - The effect of the noise is analyzed using a kinematic model
 - Remember the integrals? We need to solve them...

Sensor Models - Range finder

- The uncertainty of the scan must be evaluated
- We distinguish
 - 2D scan (n distance and angle pairs)
 - Pre-processed scan data (distance and angle to landmark(s))



Sensor Models - Range finder

- We want to model $p(z_t^k | \vec{x}_m, m)$
- What uncertainty needs to be considered?
 - **Hit:** The value of the scan (=range) is noisy: $\epsilon_{hit} \sim N(0, \sigma_{hit})$
 - **Random hit:** The probability of a random hit is modelled $U(0, \sigma_{hit})$
 - **Max. val:** The maximum value must be modeled anyhow. Thus if the range z_t^k is z_{max} , we set $p(z_t^k = z_{max} | \vec{x}_m, m) = 1$
- $p(z_t^k | \vec{x}_m, m)$ is modelled using p_{hit} , p_{random} and p_{max}
- For this, we introduce mixing parameters z_{hit} , z_{random} and z_{max}
- These mixing parameters can be learned

Likelihood Fields for Range Finders

- We now derive an algorithm to model $p(z_t^k | \vec{x}_m, m)$
- Initially, we describe the scan in the world frame
- This is done for each beam
- We use $\vec{x}$, m and $\vec{z}_t$ as inputs
- Each scan gives a set of k range-angle pairs: z_t^k and $\theta_{k,sens}$
- The sensor is mounted on $(x_{k,sens}, y_{k,sens})^T$ in the robots frame
- We convert each beam using

$$\underbrace{\begin{pmatrix} x_{z_t^k} \\ y_{z_t^k} \end{pmatrix}}_{\text{pt. of scan in world frame}} = \underbrace{\begin{pmatrix} x \\ y \end{pmatrix}}_{\text{rob. position in world frame}} + \underbrace{\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}}_{\text{rotation in world frame}} \underbrace{\begin{pmatrix} x_{k,sens} \\ y_{k,sens} \end{pmatrix}}_{\text{pos. of sensor in robot frame}} + \underbrace{z_t^k}_{\text{scan}} \underbrace{\begin{pmatrix} \cos(\theta + \theta_{k,sens}) \\ \sin(\theta + \theta_{k,sens}) \end{pmatrix}}_{\text{get x and y part of scan}}$$

(1)

Likelihood Fields for Range Finders

- $(x_{z_t^k}, y_{z_t^k})^T$ is investigated (We see if those values make sense using the robot's pose and the map)
- We check each bean and use $p(\vec{z}_t | \vec{x}_m, m) = \prod p(z_t^k | \vec{x}_m, m)$

Note before we check the algorithm:

$(x_{z_t^k}, y_{z_t^k})^T$ are numerical values coming from the scan. However, these values will typically not be the same as the map centers. Thus, we need to find the cell next to the calculated positions in the world frame. This is done by simply using the closest cell. We evaluate, how good the sensor data fits the current map and pose!

Likelihood Fields for Range Finders

```
1:   Algorithm likelihood_field_range_finder_model( $z_t, x_t, m$ ):  
2:        $q = 1$   
3:       for all  $k$  do  
4:           if  $z_t^k \neq z_{\max}$   
5:                $x_{z_t^k} = x + x_{k,\text{sens}} \cos \theta - y_{k,\text{sens}} \sin \theta + z_t^k \cos(\theta + \theta_{k,\text{sens}})$   
6:                $y_{z_t^k} = y + y_{k,\text{sens}} \cos \theta + x_{k,\text{sens}} \sin \theta + z_t^k \sin(\theta + \theta_{k,\text{sens}})$   
7:                $dist^2 = \min_{x', y'} \left\{ (x_{z_t^k} - x')^2 + (y_{z_t^k} - y')^2 \mid \langle x', y' \rangle \text{ occupied in } m \right\}$   
8:                $q = q \cdot \left( z_{\text{hit}} \cdot \text{prob}(dist^2, \sigma_{\text{hit}}^2) + \frac{z_{\text{random}}}{z_{\max}} \right)$   
9:       return  $q$ 
```

Line 2: Inits the probability which will be returned

Line 3: For loop over all beams in z_t

Line 4: Checks if beam is valid

Likelihood Fields for Range Finders

```
1:  Algorithm likelihood_field_range_finder_model( $z_t, x_t, m$ ):  
2:       $q = 1$   
3:      for all  $k$  do  
4:          if  $z_t^k \neq z_{\max}$   
5:               $x_{z_t^k} = x + x_{k,\text{sens}} \cos \theta - y_{k,\text{sens}} \sin \theta + z_t^k \cos(\theta + \theta_{k,\text{sens}})$   
6:               $y_{z_t^k} = y + y_{k,\text{sens}} \cos \theta + x_{k,\text{sens}} \sin \theta + z_t^k \sin(\theta + \theta_{k,\text{sens}})$   
7:               $dist^2 = \min_{x', y'} \left\{ (x_{z_t^k} - x')^2 + (y_{z_t^k} - y')^2 \mid \langle x', y' \rangle \text{ occupied in } m \right\}$   
8:               $q = q \cdot \left( z_{\text{hit}} \cdot \text{prob}(dist^2, \sigma_{\text{hit}}^2) + \frac{z_{\text{random}}}{z_{\max}} \right)$   
9:      return  $q$ 
```

Line 5,6: Calculates the beam's global coordinates

Line 7: Gets the closest cell to the calculated beam position (yes, its the square root)

Line 8: Applies our probabilistic model. prob is our zero-mean Gaussian.

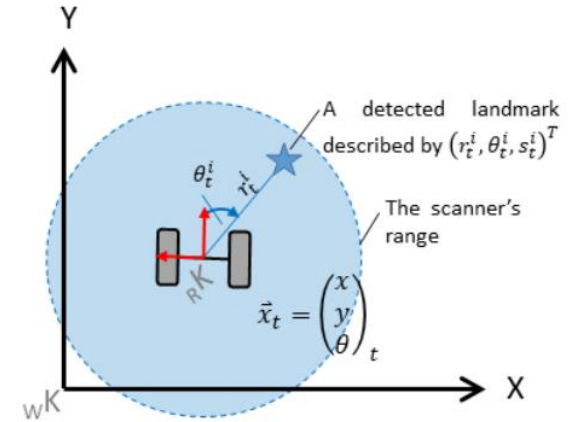
Likelihood Fields for Range Finders

- This is rather complicated and computational expensive
- Note, we can calculate all possible values before the application. We can create a lookup-table.
- However, we can pre-process the scan!
- We can extract "landmarks" from the scan and simplify the algorithm

For example, we can use the scan to locate a cylinder. Thus we can use the cylinders position and a map. Then, all other beams can be ignored. Note: We must know the correspondence!

Landmark model

- A landmark is detected in the scan
- This landmark needs to be described in the world frame.
- Initially, we will calculate pseudo sensor messages based on the map and the robot's pose.
- Afterwards, we "compare" those values to the true measurement.



Landmark model

```
1:   Algorithm landmark_model known_correspondence( $f_t^i, c_t^i, x_t, m$ ):  
2:        $j = c_t^i$   
3:        $\hat{r} = \sqrt{(m_{j,x} - x)^2 + (m_{j,y} - y)^2}$   
4:        $\hat{\phi} = \text{atan2}(m_{j,y} - y, m_{j,x} - x)$   
5:        $q = \text{prob}(r_t^i - \hat{r}, \sigma_r^2) \cdot \text{prob}(\phi_t^i - \hat{\phi}, \sigma_\phi^2) \cdot \text{prob}(s_t^i - s_j, \sigma_s^2)$   
6:       return  $q$ 
```

Each "sensor value" is here described using $\vec{f}_t^i = (r_t^i, \phi_t^i, \vec{s}_t^i)^T$

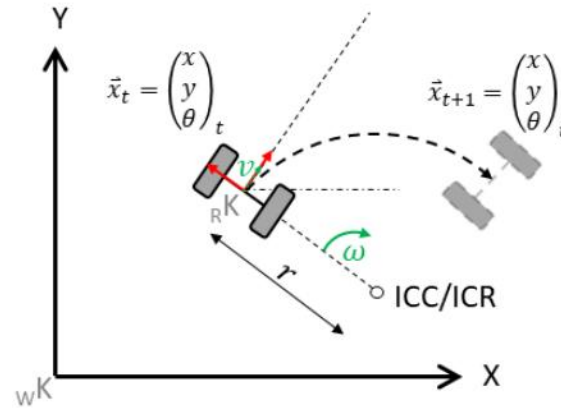
Line 2: We get the correspondence (Please see EKF Localization)

Line 3,4: We get a pseudo range and angle from the map and the $\vec{x}_t$

Line 5: Similarly, we evaluate the differences in the range, angle and signature using a distribution (e.g.: a Gaussian).

Motion Models for Mobile Robots

- We analyse the motion of a simple robot
- The robot moves in circular pattern
- The circle's center is the ICC or ICR (instantaneous center of curvature/rotation)
- But before we get a probabilistic motion model, we calculate our motion equations



Motion Models for Mobile Robots

- From basic physics we know

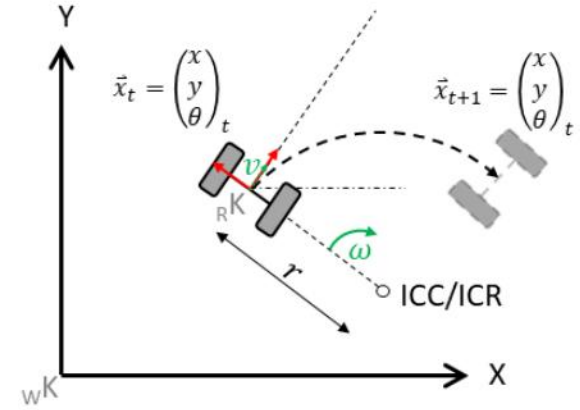
$$v = r * \omega \rightarrow r = \left| \frac{v}{\omega} \right|$$

- Using that to get the ICC's position

$$\begin{pmatrix} x_{ICC} \\ y_{ICC} \end{pmatrix} = \begin{pmatrix} x_t \\ y_t \end{pmatrix} + \frac{v}{\omega} \begin{pmatrix} -\sin(\theta) \\ \cos(\theta) \end{pmatrix}$$

- And finally

$$\vec{x}_{t+1} = \begin{pmatrix} x_{ICC} + \frac{v}{\omega} \sin(\theta_t + \omega\Delta t) \\ y_{ICC} - \frac{v}{\omega} \cos(\theta_t + \omega\Delta t) \\ \theta + \omega\Delta t \end{pmatrix} = \begin{pmatrix} x_t \\ y_t \\ \theta_t \end{pmatrix} + \begin{pmatrix} -\frac{v}{\omega} \sin(\theta_t) + \frac{v}{\omega} \sin(\theta_t + \omega\Delta t) \\ \frac{v}{\omega} \cos(\theta_t) - \frac{v}{\omega} \cos(\theta_t + \omega\Delta t) \\ \theta + \omega\Delta t \end{pmatrix}$$



Motion Models for Mobile Robots

- Where can "uncertainty" occur?

$$\vec{x}_{t+1} = \begin{pmatrix} x_{ICC} + \frac{v}{\omega} \sin(\theta_t + \omega \Delta t) \\ y_{ICC} - \frac{v}{\omega} \cos(\theta_t + \omega \Delta t) \\ \theta + \omega \Delta t \end{pmatrix} = \begin{pmatrix} x_t \\ y_t \\ \theta_t \end{pmatrix} + \begin{pmatrix} -\frac{v}{\omega} \sin(\theta_t) + \frac{v}{\omega} \sin(\theta_t + \omega \Delta t) \\ \frac{v}{\omega} \cos(\theta_t) - \frac{v}{\omega} \cos(\theta_t + \omega \Delta t) \\ \theta + \omega \Delta t \end{pmatrix}$$

- Slippage and sliding introduces noise in v and ω
- There is maybe noise at the wheel encoders

→ So we can maybe describe the "noise" at the wheels and ICC

Remember, we need a pdf for our filters:

How can we calculate the robot's pdf using the wheel noise?

Motion Models for Mobile Robots

- Where can "uncertainty" occur?

$$\vec{x}_{t+1} = \begin{pmatrix} x_{ICC} + \frac{v}{\omega} \sin(\theta_t + \omega \Delta t) \\ y_{ICC} - \frac{v}{\omega} \cos(\theta_t + \omega \Delta t) \\ \theta + \omega \Delta t \end{pmatrix} = \begin{pmatrix} x_t \\ y_t \\ \theta_t \end{pmatrix} + \begin{pmatrix} -\frac{v}{\omega} \sin(\theta_t) + \frac{v}{\omega} \sin(\theta_t + \omega \Delta t) \\ \frac{v}{\omega} \cos(\theta_t) - \frac{v}{\omega} \cos(\theta_t + \omega \Delta t) \\ \theta + \omega \Delta t \end{pmatrix}$$

- Slippage and sliding introduces noise in v and ω
- There is maybe noise at the wheel encoders

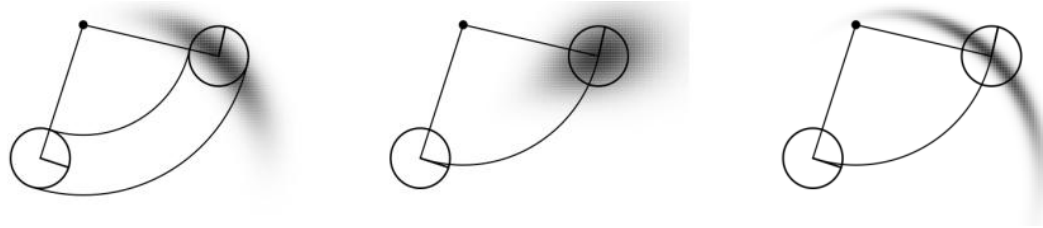
→ So we can maybe describe the "noise" at the wheels and ICC

Remember, we need a pdf for our filters:

How can we calculate the robot's pdf using the wheel noise?

Motion Models for Mobile Robots

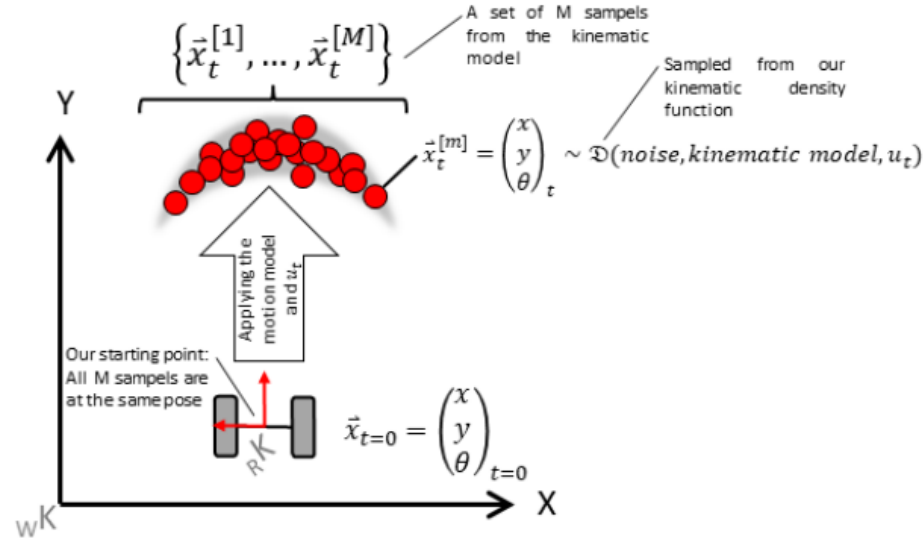
- Let us simulate the pdf using different noise settings



- It is hard to implement these pdf. How to calibrate?
- There exist a statistical toolbox for those kinds of problems:
 - Markov chain Monte Carlo (MCMC)
 - Instead of calculating parameters (μ or Σ), we sample and evaluate the samples

We will now sample from the kinematic model This is a hard statistical method.
We keep it simple. If you want to know more, please see (Bishop, 2006)

Motion Models for Mobile Robots



- We sample from the kinematic model (red dots)
- Each sample is a pose-hypothesis
- We need to formulate D , a sampling function

Motion Models for Mobile Robots

```
1:  Algorithm sample_motion_model_velocity( $u_t, x_{t-1}$ ):  
2:       $\hat{v} = v + \text{sample}(\alpha_1|v| + \alpha_2|\omega|)$   
3:       $\hat{\omega} = \omega + \text{sample}(\alpha_3|v| + \alpha_4|\omega|)$   
4:       $\hat{\gamma} = \text{sample}(\alpha_5|v| + \alpha_6|\omega|)$   
5:       $x' = x - \frac{\hat{v}}{\hat{\omega}} \sin \theta + \frac{\hat{v}}{\hat{\omega}} \sin(\theta + \hat{\omega}\Delta t)$   
6:       $y' = y + \frac{\hat{v}}{\hat{\omega}} \cos \theta - \frac{\hat{v}}{\hat{\omega}} \cos(\theta + \hat{\omega}\Delta t)$   
7:       $\theta' = \theta + \hat{\omega}\Delta t + \hat{\gamma}\Delta t$   
8:      return  $x_t = (x', y', \theta')^T$ 
```

- Line 2,3: Use signal of $\vec{x}_{t+1} = (v \quad \omega)^T$ and add random number
- Line 4: Additional rotation noise
- Line 5-7: Apply kinematic model using noisy data